Experiment 3

Aim: Apply Decision Tree and Random Forest for classification tasks

Dataset Source:
Online Shoppers Purchasing Intention Dataset:
https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	Administr	Administr	Informatic	Informatic	ProductRe	ProductRe	BounceRa	ExitRates	PageValu	SpecialDa	Month	Operating	Browser	Region	TrafficTyp	VisitorTyp	Weekend	Revenue
2	0	0	0	0	1	0	0.2	0.2	0	0	Feb	1	1	1	1	Returning	FALSE	FALSE
3	0	0	0	0	2	64	0	0.1	0	0	Feb	2	2	1	2	Returning	FALSE	FALSE
4	0	0	0	0	1	0	0.2	0.2	0	0	Feb	4	1	9	3	Returning	FALSE	FALSE
5	0	0	0	0	2	2.666667	0.05	0.14	0	0	Feb	3	2	2	4	Returning	FALSE	FALSE
6	0	0	0	0	10	627.5	0.02	0.05	0	0	Feb	3	3	1	4	Returning	TRUE	FALSE
7	0	0	0	0	19	154.2167	0.015789	0.024561	0	0	Feb	2	2	1	3	Returning	FALSE	FALSE
8	0	0	0	0	1	0	0.2	0.2	0	0.4	Feb	2	4	3	3	Returning	FALSE	FALSE
9	1	0	0	0	0	0	0.2	0.2	0	0	Feb	1	2	1	5	Returning	TRUE	FALSE
10	0	0	0	0	2	37	0	0.1	0	0.8	Feb	2	2	2	3	Returning	FALSE	FALSE
11	0	0	0	0	3	738	0	0.022222	0	0.4	Feb	2	4	1	2	Returning	FALSE	FALSE
12	0	0	0	0	3	395	0	0.066667	0	0	Feb	1	1	3	3	Returning	FALSE	FALSE
13	0	0	0	0	16	407.75	0.01875	0.025833	0	0.4	Feb	1	1	4	3	Returning	FALSE	FALSE
14	0	0	0	0	7	280.5	0	0.028571	0	0	Feb	1	1	1	3	Returning	FALSE	FALSE
15	0	0	0	0	6	98	0	0.066667	0	0	Feb	2	5	1	3	Returning	FALSE	FALSE
16	0	0	0	0	2	68	0	0.1	0	0	Feb	3	2	3	3	Returning	FALSE	FALSE
17	2	53	0	0	23	1668.285	0.008333	0.016313	0	0	Feb	1	1	9	3	Returning	FALSE	FALSE
18	0	0	0	0	1	0	0.2	0.2	0	0	Feb	1	1	4	3	Returning	FALSE	FALSE
19	0	0	0	0	13	334.9667	0	0.007692	0	0	Feb	1	1	1	4	Returning	TRUE	FALSE
20	0	0	0	0	2	32	0	0.1	0	0	Feb	2	2	1	3	Returning	FALSE	FALSE

Dataset Description: This dataset contains real-world e-commerce session data used to predict whether a visitor will make a purchase.

Dataset Overview

- 12,330 instances
- 18 input features
- 1 target variable

Features

Feature	Description
Administrative	Number of administrative pages visited
Administrative_Duration	Time spent on admin pages
Informational	Number of informational pages
Informational_Duration	Time spent on informational pages
ProductRelated	Number of product pages visited
ProductRelated_Duration	Time spent on product pages

BounceRates	Percentage of visitors leaving quickly
ExitRates	Page exit probability
PageValues	Value of the webpage
SpecialDay	Closeness to special day (e.g., holiday)
Month	Month of visit
OperatingSystems	OS used
Browser	Browser used
Region	User region
TrafficType	Traffic source
VisitorType	Returning or new visitor
Weekend	Whether visit occurred on weekend

Target Variable

Revenue

Value	Meaning
1	Customer made purchase
0	Customer did not purchase

This dataset models **real customer purchase prediction in e-commerce**.

Values:

- **True → Purchase made**
- **False → No purchase**

Real-World Application

This dataset helps **e-commerce companies predict potential buyers**, allowing them to:

- improve marketing strategies
- personalize recommendations
- increase conversion rates

3. Mathematical Formulation of Algorithms

Decision Tree

A **Decision Tree** splits the dataset into subsets based on feature values to minimize impurity. The most commonly used measure is **Gini Index**.

Gini Index

$$Gini = 1 - \sum_{i=1}^{c} p_i^2$$

Where:

- pi= probability of class iii
- c= number of classes

Lower Gini value indicates **better split**.

Entropy (Information Gain)

Another criterion is **Entropy**.

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Information Gain:

$$IG = Entropy(parent) - \sum w_i \times Entropy(child_i)$$

The feature with **maximum information gain** is selected for splitting.

Random Forest

Random Forest is an **ensemble learning method** that combines multiple Decision Trees. Each tree is trained on:

- **Random subset of data (Bootstrap Sampling)**
- **Random subset of features**

Final prediction is obtained using **majority voting**.

$$Prediction = \text{mode}(T_1(x), T_2(x), ..., T_n(x))$$

Where:

- T_n = individual trees
- x = input data point

Random Forest improves:

- accuracy
- robustness
- generalization

4. Algorithm Limitations

Decision Tree Limitations

1. **Overfitting:** Trees may become very deep and memorize training data.
2. **High Variance:** Small changes in data may produce different trees.
3. **Bias towards dominant features:** Features with many categories may dominate splits.
4. **Poor generalization:** Single tree may not perform well on unseen data.

Random Forest Limitations

1. **Higher Computational Cost:** Multiple trees increase training time.
2. **Less Interpretability:** Harder to interpret compared to a single tree.
3. **Memory Usage:** Storing many trees requires more memory.
4. **Slow Prediction:** Prediction time increases with the number of trees.

5. Methodology / Workflow

The experiment follows these steps:

Step 1 – Dataset Collection

[1]
15s

```
from google.colab import files
uploaded = files.upload()
```

Choose Files online_sho...intention.csv

online_shoppers_intention.csv(text/csv) - 1072063 bytes, last modified: 3/16/2026 - 100% done

```
Saving online_shoppers_intention.csv to online_shoppers_intention.csv
```

Step 2 – Data Preprocessing

Operations performed: Handle missing values, Convert categorical features, Normalize/scale numerical values if needed

```
 17  Revenue                  12330 non-null  bool
dtypes: bool(2), float64(7), int64(7), object(2)
memory usage: 1.5+ MB
```

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRate
count	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.00000
mean	2.315166	80.818611	0.503569	34.472398	31.731468	1194.746220	0.022191	0.04307
std	3.321784	176.779107	1.270156	140.749294	44.475503	1913.669288	0.048488	0.04859
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
25%	0.000000	0.000000	0.000000	0.000000	7.000000	184.137500	0.000000	0.01428
50%	1.000000	7.500000	0.000000	0.000000	18.000000	598.936905	0.003112	0.02515
75%	4.000000	93.256250	0.000000	0.000000	38.000000	1464.157214	0.016813	0.05000
max	27.000000	3398.750000	24.000000	2549.375000	705.000000	63973.522230	0.200000	0.20000

Step 3 – Data Splitting

Dataset is divided into: **Training set (80%)** and **Testing set (20%)**

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", X_train.shape)
print("Testing samples:", X_test.shape)
```

```
Training samples: (9864, 17)
Testing samples: (2466, 17)
```

Step 4 – Model Training

Two models are trained:

1. Decision Tree Classifier

```
dt_model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=10,
    random_state=42
)

dt_model.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=10, random_state=42)
```

2. Random Forest Classifier

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=12, n_estimators=200, random_state=42)
```

Step 5 – Prediction

Models predict the **Revenue class** for test data.

```
dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)

print("Decision Tree Accuracy:", dt_accuracy)
```

```
Decision Tree Accuracy: 0.8832116788321168
```

```
rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)

print("Random Forest Accuracy:", rf_accuracy)
```

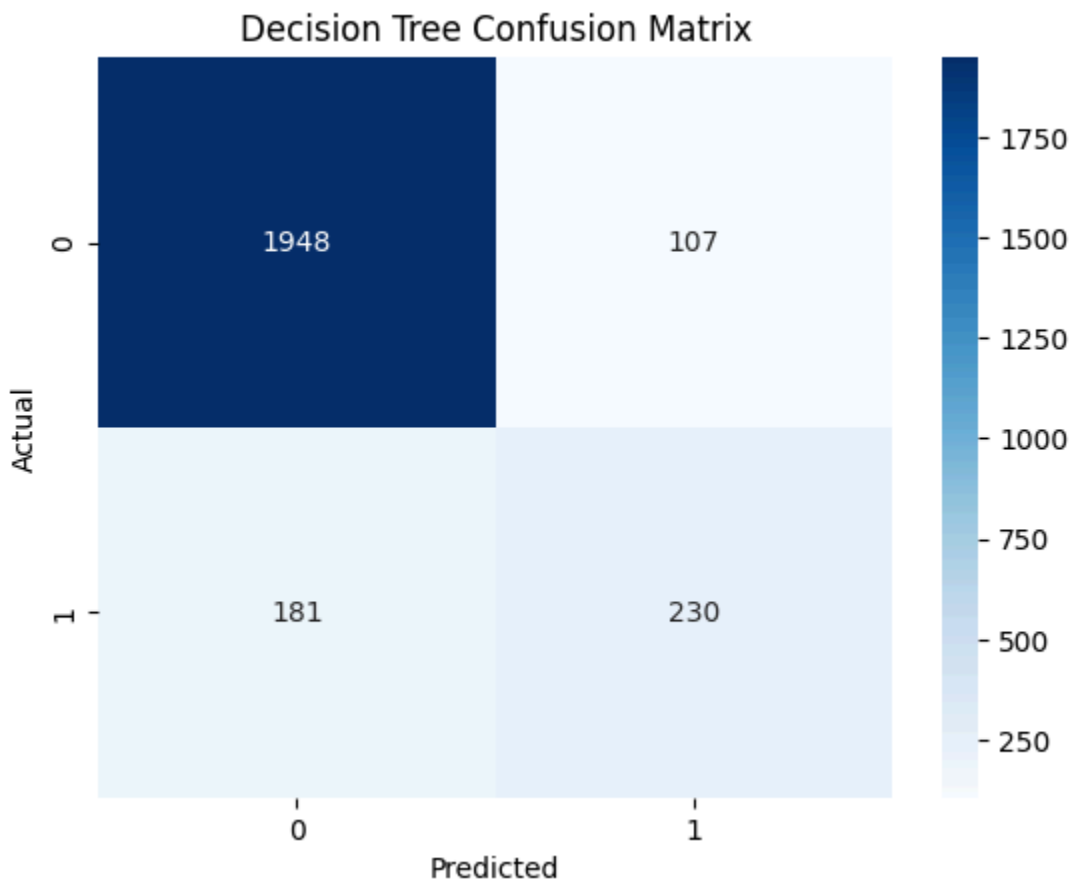
```
Random Forest Accuracy: 0.8941605839416058
```

Step 6 – Performance Evaluation

Evaluation metrics used:

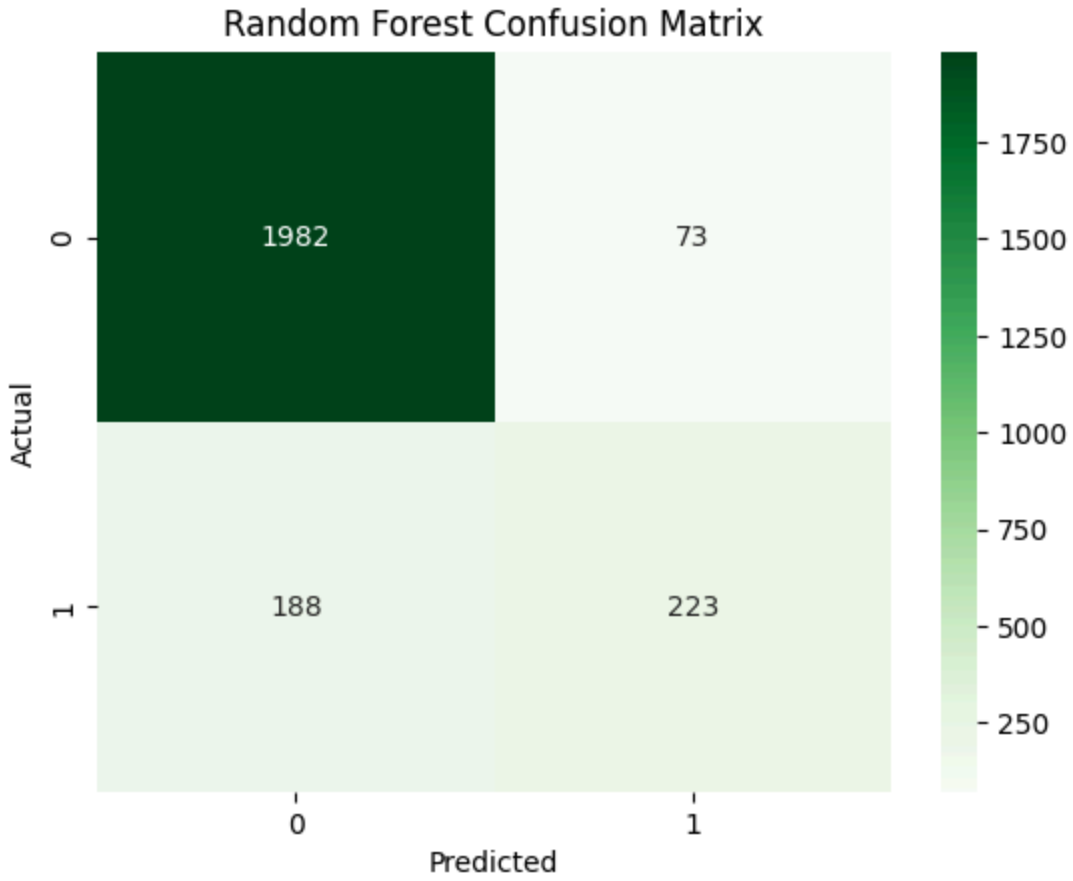
- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

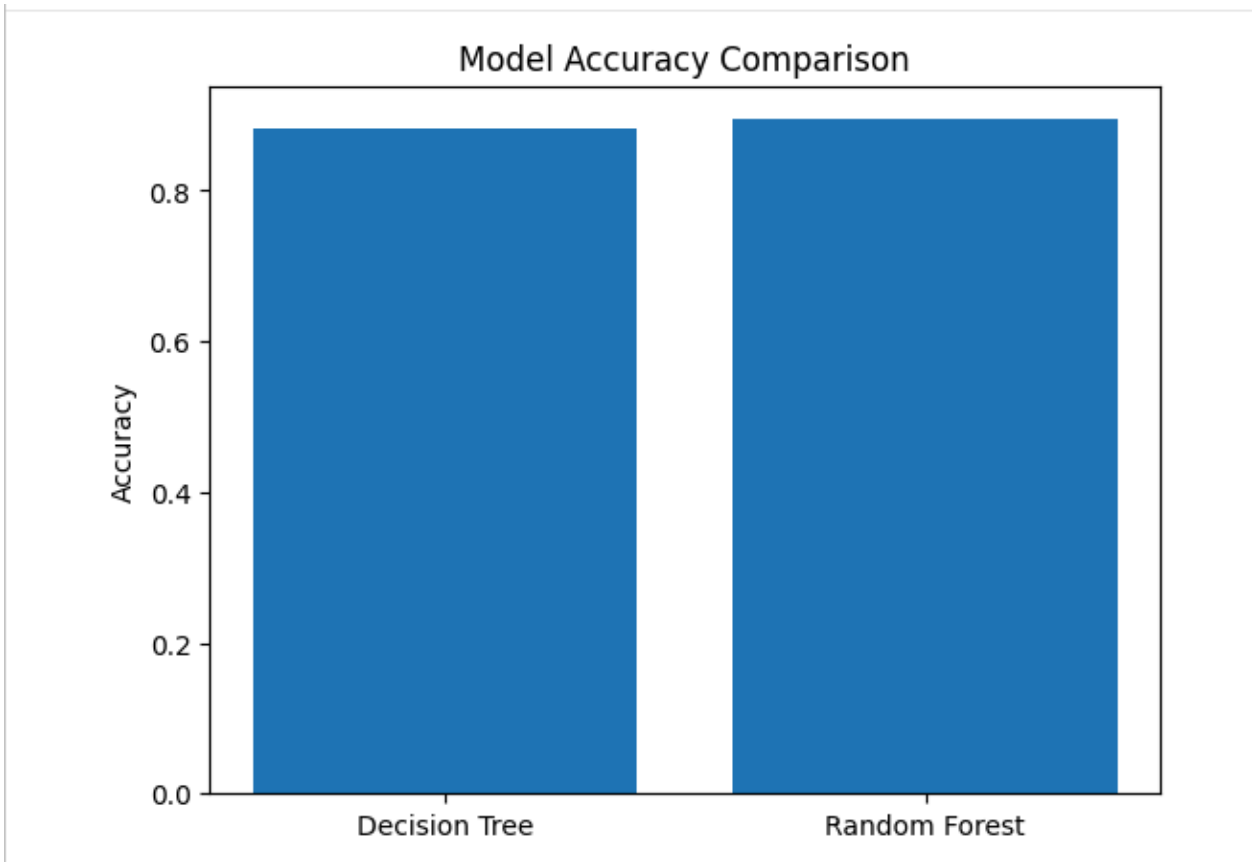
```
Classification Report

              precision    recall  f1-score   support

       False       0.91      0.95      0.93      2055
        True       0.68      0.56      0.61       411

    accuracy                           0.88      2466
   macro avg       0.80      0.75      0.77      2466
weighted avg       0.88      0.88      0.88      2466
```



Classification Report

	precision	recall	f1-score	support
False	0.91	0.96	0.94	2055
True	0.75	0.54	0.63	411
accuracy			0.89	2466
macro avg	0.83	0.75	0.78	2466
weighted avg	0.89	0.89	0.89	2466


Random Forest Confusion Matrix
1982
73
188
223
0
1
Actual
0
1
Predicted
1750
1500
1250
1000
750
500
250



6. Performance Analysis

Performance is evaluated using classification metrics.

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall

$$Recall = \frac{TP}{TP + FN}$$

F1 Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Expected observation:

Model	Accuracy
Decision Tree	~88%
Random Forest	~92–94%

Random Forest usually performs better because it **reduces overfitting using multiple trees**.

7. Hyperparameter Tuning

Hyperparameter tuning improves model performance.

Decision Tree Parameters

Parameter	Description
max_depth	Maximum depth of tree
min_samples_split	Minimum samples required to split node
min_samples_leaf	Minimum samples at leaf node
criterion	Split method (gini / entropy)

Random Forest Parameters

Parameter	Description
n_estimators	Number of trees
max_depth	Depth of trees
max_features	Number of features used per split
min_samples_split	Minimum samples required to split

Tuning Method

Common tuning methods:

- Grid Search
- Random Search

- Cross Validation

Hyperparameter tuning improves **accuracy and generalization** of the model.